

TERUG NAAR MAKER MANUALS

MAKER MANUALS 01

VIBECODING

Apps en tools bouwen zonder technische achtergrond.

Formaat

Webeditie in neo brutalism-stijl

Licentie

CC BY-SA 4.0

Status

Alpha-editie 0.8

Alpha-editie 0.8 - levend boek in ontwikkeling

Dit boek is bewust een werk in uitvoering: geen eindpunt, maar een versie die beter wordt terwijl het onderwerp beweegt.

ONDERTITEL

Een praktische gids voor mensen die willen bouwen met AI zonder klassieke programmeur te zijn.

COLOFON

Gebaseerd op de nieuwsbrieven van Erwin Blom, met hulp van AI omgezet tot een `Maker Manual`.

Deze tekst is beschikbaar onder de Creative Commons-licentie `CC BY-SA 4.0`.

Dat betekent kort gezegd: je mag deze tekst delen, hergebruiken en bewerken, ook commercieel, zolang je Erwin Blom als bron noemt en afgeleide versies onder dezelfde licentie deelt.

INLEIDING

Dit boek is niet geschreven om de hipste tools van dit moment op een rij te zetten. Over een halfjaar ziet die lijst er weer anders uit.

Het is ook niet geschreven om iedereen alsnog programmeur te maken.

Het onderwerp is praktischer en interessanter tegelijk. Vibecoding begint vaak met iets kleins: een tool voor je eigen werk, een eerste prototype, een workflowhulp, een site, een oplossing voor een irritatie die te lang bleef liggen. Onder dat soort kleine ingrepen zit een grotere verschuiving. Veel meer mensen kunnen dichter op software gaan zitten dan vroeger. Ze hoeven niet meer alleen te bedenken wat ze willen; ze kunnen ook sneller iets laten ontstaan, testen en bijsturen.

In de hypelaag van dit onderwerp gaat het vaak over snelheid, wonderen en nieuwe interfaces. In dit boek kijk ik liever naar wat daaronder gebeurt.

Wat is vibecoding precies? Waarom doet het ertoe? Hoe begin je goed? Hoe voorkom je dat je verdrinkt in tools? Hoe ga je van idee

naar eerste bruikbare versie? Waarom worden smaak en oordeel juist belangrijker? En wat blijft overeind als de opwindning van het moment is gezakt?

Dat zijn de vragen van dit boek.

Ik schrijf het voor mensen die niet uit een traditionele programmeerwereld komen, maar wel ideeën, smaak, richting of praktijkkennis hebben. Schrijvers, ontwerpers, ondernemers, docenten, redacteurs, muzikanten, onderzoekers, makers. Mensen die voelen dat software dichterbij is gekomen, maar nog zoeken naar een houding, een werkwijze en woorden die minder naar hype ruiken.

De hoofdgedachte van dit boek is eenvoudig. Vibecoding maakt software voor veel meer mensen maakbaar. Daardoor verschuift het verschil tussen middelmaat en kwaliteit. Minder hangt af van pure uitvoering, meer van kijken, kiezen, sturen, vereenvoudigen, beoordelen en volhouden.

Als dit boek goed werkt, levert het rust en maaklust op. Rust, omdat je scherper ziet waar vibecoding wel en niet goed voor is. Maaklust, omdat de afstand tussen idee en eerste versie kleiner voelt zodra je met de juiste verwachtingen begint.

Dat is genoeg ambitie voor één manual.

INHOUD

1. Waarom vibecoding ertoe doet
2. Wat vibecoding wel en niet is
3. Begin kleiner dan je denkt
4. Werk vanuit taken, niet vanuit tools
5. Van idee naar eerste bruikbare versie

6. Smaak, oordeel en productgevoel
7. Het speelveld verandert snel, dus jouw houding moet meebewegen
8. Wat blijft als de hype weg is

HOOFDSTUK 1 - WAAROM VIBECODING ERTOE DOET

Sommige veranderingen ogen eerst als een aardige truc en blijken pas later een serieuze verschuiving. Vibecoding hoort in die categorie.

Op het eerste gezicht is het vooral spectaculair. Je beschrijft in gewone taal wat je wilt maken, een systeem gaat aan het werk en even later staat er een site, tool of app voor je neus. Als je geen programmeur bent, voelt dat al snel onwerkelijk.

De betekenis zit in het bereik. Software komt binnen handbereik van veel meer mensen dan voorheen: schrijvers, ontwerpers, docenten, muzikanten, redacteurs, zelfstandigen, onderzoekers en ondernemers die vroeger wel ideeën over software hadden en voor de uitvoering vrijwel altijd afhankelijk waren van anderen.

Niet iedereen wordt programmeur. Veel meer mensen worden wel maker.

DE AFSTAND TUSSEN IDEE EN UITVOERING WORDT KLEINER

Tot voor kort zag het proces er vaak zo uit: je had een idee, beschreef dat zo goed mogelijk, stuurde het door naar iemand die technisch was, wachtte op een eerste versie, legde uit wat er niet klopte, wachtte opnieuw en hoopte dat het eindresultaat nog een beetje in de buurt kwam van wat je in je hoofd had.

Dat model verdwijnt niet helemaal. Grote systemen, complexe producten en serieuze softwareteams blijven nodig. Maar er is iets fundamenteels veranderd aan de onderkant en in het midden van de markt. Voor heel veel ideeën is de afstand tussen “ik zie iets voor me” en “ik kan een eerste versie laten ontstaan” veel kleiner geworden.

Dat heeft twee gevolgen.

De eerste is praktisch. Je kunt sneller testen of een idee ergens op slaat. Je hoeft minder lang in abstracties te blijven hangen. Je ziet eerder waar het schuurt. Je leert sneller wat je eigenlijk bedoelt.

De tweede is psychologisch. Zodra mensen merken dat iets maakbaar wordt, gaan ze anders denken. Dan verschuift de vraag van “zou dit kunnen?” naar “wat zou ik hier eigenlijk van willen maken?”

Precies daar begint een andere mentaliteit.

NIET-TECHNISCHE MAKERS KRIJGEN EEN NIEUWE ROL

Vroeger hoorde software maken voor veel mensen bij een andere wereld. Je kon er wel iets van vinden, iets voor wensen of een briefing over schrijven, maar de maaklaag lag buiten bereik. Net zoals drukpersen, montagesets of studio's ooit voorbehouden waren aan specialisten en instellingen.

De vergelijking met desktop publishing of het vroege web helpt hier. Niet omdat vibecoding hetzelfde is, wel omdat je opnieuw ziet wat er gebeurt als een productiemiddel toegankelijker wordt. Zodra gereedschap goedkoper, sneller en bereikbaarder wordt, verschuift vooral wie er mee kan doen.

Dat zie je nu met software.

Een ontwerper hoeft niet meer alleen een mockup te maken. Een redacteur hoeft niet meer alleen te beschrijven hoe een tool voor research of publicatie eruit zou moeten zien. Een zelfstandige hoeft niet meer te blijven hangen in frustratie over software die bijna goed is, maar net niet past. Er ontstaat een nieuwe tussenvorm tussen volledig afhankelijk zijn van anderen en alles zelf technisch beheersen.

Die tussenvorm is belangrijk.

Want juist daar zitten de mensen met veel ideeën, veel praktijkkennis en veel gevoel voor wat er ontbreekt, maar zonder klassieke programmeerachtergrond. Vibecoding geeft die groep geen toverstok. Wel een nieuwe rol.

Niet de rol van engineer. Wel de rol van maker-regisseur.

DE REVOLUTIE SPEELT ZICH DICHTBIJ HUIS AF

Dit onderwerp wordt snel abstract. Voor je het weet praat je over democratisering en makerkracht zonder nog iets voor je te zien.

De afgelopen tijd zag ik meerdere voorbeelden langskomen van mensen die geen klassieke programmeur zijn, maar wel gewoon software gingen bouwen zodra ze iets nodig hadden.

Jaap van Zessen bouwde in een paar uur een Chrome-extensie voor het WK voetbal, omdat hij 's ochtends samenvattingen zonder spoilers wilde kunnen kijken.

Richard Theuws werkt aan een communitysite voor zijn Metal Business Club, omdat hij voor zijn eigen club iets wilde dat beter paste dan wat er al bestond.

Bjorn Bouwmeester wilde zijn zoon helpen in het eindexamenjaar en bouwde een site om toetsen mee te trainen.

Zara Zhang bouwt browser-extensies en andere tools rond haar eigen werkwijze, bijvoorbeeld om lange video's sneller terug te brengen tot de belangrijkste lessen.

Thom Easix bouwde op een avond zijn eigen versie van Google Meet, aangesloten op de audiovisuele tools die hij zelf al gebruikte.

Het punt van deze voorbeelden is niet dat ze indrukwekkend klinken in een opsomming. Ze laten zien hoe dicht software maken ineens op het dagelijkse werk van niet-programmeurs is komen te zitten. Deze makers wachtten niet op budget, een team of toestemming. Ze zagen iets wat beter kon en gingen aan de slag.

Daarom voelt het woord revolutie hier niet overdreven. De macht om software te maken schuift zichtbaar op naar veel meer mensen die een probleem dichtbij huis zien en het zelf kunnen proberen op te lossen.

DE BOTTLENECK VERSCHUIFT

Als software bouwen toegankelijker wordt, betekent dat niet dat alles vanzelf goed wordt. Het betekent dat de schaarste ergens anders komt te liggen.

De bottleneck zit dan minder in pure technische uitvoering en meer in:

- zien wat er echt nodig is
- een probleem scherp formuleren
- goede vragen stellen
- richting houden tijdens iteraties
- smaak toepassen
- kunnen beoordelen of iets bruikbaar is
- weten wanneer je moet stoppen, doorgaan of opnieuw beginnen

Dat zijn geen kleine dingen. Dat zijn precies de menselijke vermogens die een verschil maken zodra de drempel tot uitvoering daalt.

Veel mensen denken bij vibecoding nog steeds: mooi, dan doet de machine het zware werk en is het probleem opgelost. Maar in de praktijk zie je iets anders. Zodra de techniek minder exclusief wordt, worden oordeel, smaak en productgevoel juist belangrijker.

Een tool kan iets genereren. Maar hij weet niet vanzelf wat voor jou relevant is. Hij voelt niet wat overbodig is. Hij herkent niet automatisch de juiste toon. Hij begrijpt niet uit zichzelf wat voor gebruiker je voor ogen hebt. Hij maakt geen afweging tussen snelheid, helderheid, vertrouwen en kwaliteit zoals jij dat als mens kunt doen.

Vibecoding maakt software dus niet simpel.

Vibecoding maakt software maakbaar.

En dat is iets heel anders.

KLEIN WORDT INTERESSANTER

Nog een reden waarom vibecoding ertoe doet: het maakt kleine software economisch en praktisch interessanter.

In de oude logica moest software al snel groot genoeg zijn om de kosten van ontwerp en ontwikkeling te rechtvaardigen. Daardoor kreeg je vaak generieke producten voor brede doelgroepen. Dat is logisch. Maar het betekende ook dat allerlei kleine, specifieke of persoonlijke wensen nooit de moeite waard leken om te bouwen.

Nu ineens wel.

Een tool voor je eigen research. Een interne workflowhulp voor een klein team. Een nicheplatform voor een heel specifieke community.

Een tijdelijke site rond een project. Een hulpmiddel dat maar één taak doet, maar die taak perfect genoeg uitvoert.

Dat soort software was vroeger vaak te klein, te duur of te omslachtig. Nu niet meer. Daardoor wordt klein maar raak een veel interessantere categorie.

Dat sluit aan op een bredere culturele beweging die me al langer fascineert. We hebben lang gedacht dat groter automatisch beter was: meer functies, meer gebruikers, meer platform, meer bereik. Maar veel van de interessantste dingen van dit moment zijn juist kleiner, specifieker en persoonlijker. Niet alles hoeft een alles-in-één systeem te zijn. Niet alles hoeft op te schalen naar de hele wereld om waardevol te zijn.

Vibecoding past in die beweging.

Het maakt ruimte voor software die dichter op een specifieke maker, taak of context zit.

DIT GAAT NIET ALLEEN OVER TOOLS

Wie alleen naar vibecoding kijkt als een verzameling handige tools, onderschat wat er aan de hand is. Tools zijn belangrijk, natuurlijk. Maar ze zijn niet het verhaal. Ze zijn de tijdelijke vorm waarin iets groters zichtbaar wordt.

Het grotere verhaal is dat software verschuift van een domein waar je vooral iets van kon vragen naar een domein waar je veel vaker zelf iets in gang kunt zetten.

Dat betekent niet dat elke vibecoder goede software maakt. Zeker niet. We zullen ook een golf van rommel, middelmaat en overschatting zien. Dat hoort erbij. Elke democratisering van maken levert naast bevrijding ook ruis op.

Maar dat bezwaar raakt de kern niet.

Toen veel meer mensen ineens een boek konden drukken, leverde dat ook niet alleen goede boeken op. Hetzelfde gold later voor fotografie en websites. Toegang vergroot de kans op kwaliteit, maar garandeert haar niet.

Toegankelijkheid garandeert geen kwaliteit.

Maar het vergroot wel de kans dat nieuwe stemmen, nieuwe vormen en nieuwe oplossingen ontstaan.

En juist daarom is vibecoding belangrijk.

DE NIEUWE VRAAG

Als je eenmaal ziet wat hier verschuift, verandert ook de vraag die je jezelf stelt.

Niet langer alleen:

kan dit gebouwd worden?

Maar steeds vaker:

wat wil ik eigenlijk bouwen?

En:

voor wie?

En:

waarom bestaat dit nog niet?

En:

wat is de kleinste bruikbare versie waarmee ik vandaag kan beginnen?

Dat zijn maker-vragen. Geen tool-vragen.

Het interessante aan vibecoding is dus niet alleen dat machines sneller kunnen bouwen. Het interessante is dat meer mensen zich serieus gaan gedragen als iemand die mag maken.

Daar zit voor mij de echte belofte.

Niet in perfectie. Niet in automatisch succes. Niet in het verdwijnen van vakmanschap.

Maar in het feit dat ideeën minder vaak hoeven te stranden in de fase ervoor.

WAT DIT HOOFDSTUK MOET LATEN STAAN

Als je na dit hoofdstuk maar één ding onthoudt, laat het dan dit zijn:

Vibecoding is belangrijk omdat veel meer mensen software kunnen benaderen als iets wat binnen hun bereik ligt.

De wereld wordt daar niet automatisch beter van. Wel rijker aan mogelijke makers.

HOOFDSTUK 2 - WAT VIBECODING WEL EN NIET IS

Hoe populairder een nieuw begrip wordt, hoe groter de kans dat het uitrekt.

Dat zie je nu ook met vibecoding.

Voor de een betekent het: met gewone taal software maken. Voor de ander: snel prototypes bouwen met AI. Voor weer iemand anders: agents op elkaar stapelen, workflows automatiseren of simpelweg een beetje chatten met een tool die ook code kan schrijven.

Er zit iets bruikbaars in al die omschrijvingen, maar ook veel ruis.

Daarom is het goed om vroeg in dit boek iets scherper te worden.

Niet om een hip woord te bewaken. Wel om verkeerde verwachtingen te voorkomen.

Want als je niet precies genoeg weet waar vibecoding over gaat, ga je er te veel of juist te weinig van verwachten.

VIBECODING IS EEN MAAKVORM

De definitie die ik in dit boek het nuttigst vind, is deze:

Vibecoding is een manier van bouwen waarbij je met taal, feedback en iteratie software laat ontstaan zonder dat je alles vooraf technisch hoeft uit te schrijven.

Die definitie heeft vier onderdelen.

1. Bouwen Het gaat niet alleen om ideeën, analyses of gesprekken. Er moet iets ontstaan.
1. Taal Je stuurt niet alleen met code, maar ook met gewone taal, instructies, correcties en richting.
1. Feedback Je kijkt naar wat er terugkomt, beoordeelt dat en stuurt bij.
1. Iteratie Je komt zelden in één keer op de juiste plek. Het proces is een lus, geen rechte lijn.

Samen maken die vier onderdelen vibecoding concreter dan "AI gebruiken".

VIBECODING IS NIET HETZELFDE ALS CHATTEN

Veel mensen verwarren vibecoding met slim praten tegen een model.

Dat is begrijpelijk, want de interface lijkt er vaak op. Je typt iets in, krijgt iets terug, reageert weer. Maar chatten op zichzelf is nog geen bouwen.

Je kunt prima een uur met een AI praten over een app zonder ook maar een millimeter dichterbij een bruikbare versie te komen.

Vibecoding begint pas echt wanneer taal richting geeft aan iets dat vorm krijgt.

Dus niet alleen:

- wat zou er mogelijk zijn?
- hoe zou zo'n product eruit kunnen zien?
- wat zijn interessante functies?

Maar ook:

- bouw een eerste versie
- vereenvoudig deze interface
- haal deze stap weg
- voeg alleen dit onderdeel toe
- maak dit bruikbaar voor deze taak

Dat verschil lijkt klein, maar het bepaalt of je aan het praten bent of aan het bouwen.

VIBECODING IS OOK NIET HETZELFDE ALS AUTOMATISEREN

Nog zo'n verwarring: alles wat met AI en werkverlichting te maken heeft, wordt al snel op één hoop gegooid.

Maar er is verschil tussen:

- iets laten schrijven
- een handeling laten automatiseren
- een workflow laten lopen
- een tool of interface laten bouwen

Die dingen overlappen soms, maar zijn niet hetzelfde.

Als je een terugkerende taak laat afhandelen door een script of een automation, ben je niet automatisch aan het vibecoden. Als je een agent meerdere stappen achter elkaar laat zetten, ook niet per se.

Vibecoding zit specifiek in het proces waarin je een digitaal ding laat ontstaan dat iemand kan gebruiken, testen of verder ontwikkelen, terwijl jij dat proces via taal en iteratie aanstuurt.

Dat kan klein zijn. Een simpele tool. Een minimale app. Een webpagina met logica. Een interface rond een taak.

Het hoeft niet groot te zijn om echt bouwen te zijn.

VIBECODING IS GEEN MAGISCHE KNOP

Dit moet ook duidelijk gezegd worden, omdat de buitenkant soms iets anders suggereert.

Vibecoding is geen systeem waarbij je één keer iets slims typt en daarna vanzelf een goed product krijgt.

Het is geen toverformule. Geen gegarandeerde versneller. Geen vervanging van nadenken.

Het is eerder een nieuwe samenwerking tussen mens en machine waarin de machine veel sneller kan uitvoeren dan vroeger, maar de mens nog steeds moet:

- kiezen
- begrenzen
- bijsturen
- beoordelen
- herbeginnen als dat nodig is

Wie vibecoding benadert als een jackpot, raakt vaak teleurgesteld. Wie het benadert als een maakvorm, leert veel sneller.

VIBECODING ZIT TUSSEN VOLLEDIG ZELF CODEREN EN VOLLEDIG UITBESTEDEN IN

Dat tussengebied is belangrijk.

Aan de ene kant heb je klassiek softwarewerk, waarin je zelf of als team precies uitwerkt wat er gebouwd moet worden en hoe. Aan de andere kant heb je volledig uitbesteden: jij beschrijft het, iemand anders bouwt het.

Vibecoding opent een tussenvorm.

Je bent niet alleen opdrachtgever. Je bent ook niet per se klassieke programmeur.

Je zit er middenin.

Je:

- bepaalt de richting
- ziet sneller wat niet klopt
- kunt meteen iets laten aanpassen
- hoeft minder te wachten tussen idee en uitvoering
- kunt zelf blijven sturen zonder elk detail technisch te beheersen

Dat is precies waarom vibecoding voor zoveel niet-traditionele makers interessant is.

DE JUISTE HOUDING IS MINDER HEROISCH DAN VEEL MENSEN DENKEN

Omdat het woord vibecoding een zekere lichtheid heeft, denken sommige mensen dat het vooral spontaan, intuïtief en losjes moet zijn. Alsof de “vibe” het belangrijkste is.

Maar de beste vibecoding die ik zie, is meestal minder achteloos dan het woord suggereert.

Goede makers werken niet op pure vibe. Ze werken op:

- aandacht
- richting
- correctie
- begrenzing
- herhaling

Ja, er zit speelsheid in. Ja, er zit snelheid in. Ja, je kunt verrassende sprongen maken.

Maar zonder discipline wordt het al snel ruis.

WANNEER VIBECODING GOED PAST

Vibecoding is vooral sterk als:

- je snel van idee naar eerste versie wilt
- je een kleine of middelgrote tool wilt verkennen
- je een workflow zichtbaar of bruikbaar wilt maken
- je een persoonlijke of niche-oplossing wilt bouwen

- je iteratief wilt werken en onderweg leert wat het project nodig heeft

Juist daar zit de kracht: lage frictie, snelle terugkoppeling, veel directer contact tussen intentie en uitvoering.

WANNEER VIBECODING MINDER GOED PAST

Er zijn ook situaties waarin vibecoding niet de ideale hoofdroute is.

Bijvoorbeeld als:

- je iets extreem robuust, bedrijfskritisch of zwaar gereguleerd moet bouwen
- de eisen vooraf al volledig vastliggen
- de foutmarge heel klein is
- meerdere lagen architectuur, beveiliging en governance het belangrijkste zijn
- het project groter is dan een iteratief maakproces in deze vorm goed aankan

Dat betekent niet dat AI daar geen rol kan spelen. Wel dat vibecoding dan niet vanzelf de beste aanpak is.

Het is dus geen universele methode voor alle software. Het is een sterke methode voor een bepaald type softwarewerk.

BEGRIPPENRUIS MAAKT VERKEERDE VERWACHTINGEN

In de AI-wereld schuiven woorden snel.

Agents, assistenten, copilots, builders, automations, workflows, memory, orchestration, apps, sites, tools. De labels veranderen bijna per maand. En elk label trekt zijn eigen beloftes mee.

Dat is precies waarom je jezelf moet trainen in begripsdiscipline.

Vraag steeds:

- wat doet dit nu echt?
- wat voor soort taak lost dit op?
- waar zit de autonomie wel en niet?
- bouw ik hier iets, automatiseer ik iets of verken ik iets?

Zodra je dat onderscheid maakt, wordt het speelveld een stuk minder mistig.

VIBECODING IS GEEN IDENTITEIT, MAAR EEN PRAKTIJK

Nog een kleine correctie die nuttig is: je hoeft jezelf niet per se een vibecoder te noemen om ermee te werken.

Voor sommige mensen is dat woord energiek en bevrijdend. Voor anderen klinkt het te modieus. Dat maakt niet veel uit.

Belangrijker is dat je snapt wat de praktijk inhoudt.

Niet een identiteit. Niet een scene. Niet een badge.

Maar een manier van werken waarin je met taal, iteratie en oordeel sneller iets kunt bouwen dan voorheen mogelijk was.

DE VOLWASSEN DEFINITIE

Als ik het aan het einde van dit hoofdstuk nog een keer helemaal strak moet zeggen, dan is dit de versie die ik zou laten staan:

Vibecoding is iteratief software maken met hulp van AI, waarbij taal dient om richting te geven aan iets dat gebouwd, getest en verbeterd wordt.

Dat is breed genoeg om bruikbaar te zijn. En smal genoeg om niet alles te betekenen.

WAT MOET BLIJVEN HANGEN

Als je na dit hoofdstuk maar één ding meeneemt, laat het dan dit zijn:

Vibecoding is geen hypewoord voor alles wat met AI en software te maken heeft.

Het is een specifieke maakvorm.

Niet alleen praten. Niet alleen automatiseren. Niet alleen experimenteren.

Maar via taal en iteratie iets laten ontstaan dat bruikbaar genoeg is om op verder te bouwen.

HOOFDSTUK 3 - BEGIN KLEINER DAN JE DENKT

Vibecoding maakt overmoedig. Dat is een deel van de charme en ook een risico.

Zodra je merkt dat je met gewone taal een site, tool of app kunt laten ontstaan, schiet je hoofd vooruit. Een idee dat gisteren nog theoretisch was, voelt ineens maakbaar. Voor je het weet werk je niet aan een eerste stap, maar aan een platform, een startup, een compleet systeem of een oplossing voor ongeveer alles tegelijk.

Dat is begrijpelijk en meestal een fout. Het probleem is niet ambitie, maar schaal. In deze maakvorm leer je sneller als je kleiner begint dan je ego op dat moment prettig vindt.

Wie goed wil beginnen met vibecoding, moet niet eerst groot durven dromen. Die moet eerst klein durven kiezen.

DE KLASSIEKE BEGINNERSFOUT

De grootste beginnersfout is bijna altijd dezelfde: te veel tegelijk willen.

Je wilt:

- een mooi ontwerp
- een goed werkende interface
- meerdere gebruikersrollen
- slimme AI in de app zelf
- opslag
- logins
- een mobiel goed werkende versie
- een nette huisstijl
- en liefst ook meteen een businessmodel

Dat is geen eerste project.

Dat is een recept voor frustratie.

Niet omdat het onmogelijk is. Soms kom je verrassend ver. Maar omdat je in het begin nog niet goed genoeg ziet welk deel van het probleem eigenlijk het belangrijkste is. Je hebt nog niet geleerd hoe scherp je een opdracht moet formuleren. Je weet nog niet goed waar een tool sterk of zwak in is. En je hebt nog geen gevoel voor wanneer je moet doorpakken en wanneer je beter opnieuw kunt beginnen.

Daardoor loop je het risico dat alles tegelijk onduidelijk wordt:

- was het idee niet goed?

- was de opdracht te vaag?
- koos je de verkeerde tool?
- vroeg je te veel?
- zit het probleem in de logica of in de vorm?

Als alles tegelijk beweegt, leer je weinig.

KLEIN BEGINNEN IS GEEN GEBREK AAN AMBITIE

Veel mensen horen “begin klein” als een rem. Alsof je je ideeën moet temperen. Alsof je vooral niet te veel moet willen. Maar zo bedoel ik het niet.

Klein beginnen is geen afzwakken van ambitie.

Het is de snelste route naar echte vooruitgang.

Een kleine eerste stap helpt je namelijk precies die dingen te leren die later ook nodig zijn in grotere projecten:

- hoe je een probleem scherp formuleert
- hoe je een opdracht opbouwt
- hoe je iets beoordeelt op bruikbaarheid
- hoe je bijstuurt zonder te verdwalen
- hoe je ziet of je op het verkeerde spoor zit

Dat leer je niet door meteen een monsterproject op de tool los te laten. Dat leer je door een eerste ronde overzichtelijk te houden.

Wie klein begint, leert groot.

KIES JE EERSTE PROJECT ALSOF HET EEN OEFENING IS

Een goed eerste vibecodingproject heeft vier kenmerken.

Het is:

- beperkt in scope
- direct bruikbaar
- makkelijk te beoordelen
- niet afhankelijk van vijf systemen tegelijk

Dat klinkt saai. En dat is juist de bedoeling.

Een eerste project hoeft niet indrukwekkend te zijn. Het moet leerzaam zijn.

Een eenvoudige tool voor je eigen research. Een kleine site met één duidelijke functie. Een slimme checklist. Een intern hulpmiddel voor een terugkerende taak. Een persoonlijke tool die een irritatie oplost.

Dat zijn vaak veel betere eerste projecten dan een “complete community-app”, “een AI-platform voor iedereen” of “een tool die alles overneemt wat ik normaal op een dag doe”.

Goede eerste projecten zijn niet per se klein in waarde. Ze zijn klein in oppervlak.

JE EERSTE SUCCES MOET TASTBAAR ZIJN

In het begin heb je iets nodig dat werkt.

Niet perfect. Niet groots. Niet glanzend.

Werkend.

Dat eerste tastbare succes is belangrijker dan veel mensen denken. Zodra je ziet dat er iets ontstaat waar je echt op kunt klikken, iets dat echt een handeling ondersteunt of een probleem

verkleint, verandert je verhouding tot vibecoding. Dan wordt het niet langer een interessant verhaal over wat er misschien mogelijk is. Dan wordt het iets dat je zelf kunt inzetten.

Zo'n eerste succes doet drie dingen:

- het geeft vertrouwen
- het maakt je kritischer
- het leert je concreter kijken

Vertrouwen, omdat je merkt dat de drempel echt lager ligt dan vroeger.

Kritiek, omdat je na die eerste euforie beter gaat zien wat er nog niet klopt.

Concreet kijken, omdat je aandacht verschuift van “wat vet dat dit kan” naar “waarom werkt dit stuk wel en dat andere niet?”

Precies daar begint echt leren.

KORTE FEEDBACKLUSSEN VERSLAAN GROOTSE PLANNEN

Vibecoding werkt het best als je de lus kort houdt.

Niet:

ik schrijf één enorme opdracht en hoop dat het systeem alles in één keer goed begrijpt.

Wel:

- ik beschrijf het probleem
- ik kies de kleinste bruikbare versie
- ik kijk wat eruit komt
- ik stuur gericht bij

- ik test opnieuw

Dat lijkt misschien minder heroïsch. Maar het is effectiever.

Wie korte feedbacklussen gebruikt, leert sneller waar een project in vastloopt. Soms zit het probleem in de opdracht. Soms in de tool. Soms in het idee zelf. Soms in je eigen onduidelijkheid. Juist daarom moet je niet te veel lagen tegelijk willen bouwen.

In het begin wil je snel kunnen zien:

- klopt de logica?
- klopt de richting?
- lost dit iets op?
- wil ik hiermee verder?

Daar heb je geen groot project voor nodig. Daar heb je een korte lus voor nodig.

FRUSTRATIE HOORT ERBIJ

Dit is ook het punt waarop het goed is iets onaantrekkelijks hardop te zeggen: vibecoding is niet alleen maar leuk.

Soms luisteren tools slecht. Soms bouwen ze iets anders dan je vroeg. Soms fixen ze een bug en slopen ze ondertussen drie andere dingen. Soms verlies je een uur aan iets dat achteraf in tien minuten opnieuw had gekund.

Dat hoort erbij.

Beginners schrikken daar vaak van, omdat de belofte van vibecoding aan de buitenkant zo soepel oogt. Alsof je alleen maar helder hoeft te typen en de rest vanzelf valt. Maar in de praktijk is het eerder een gesprek met een capabele, snelle, soms briljante

en soms eigenwijze uitvoerder die niet vanzelf begrijpt wat jij bedoelt.

Wie dat accepteert, wordt beter.

Wie denkt dat elke frictie een bewijs is dat het systeem faalt, haakt vaak te vroeg af.

Frustratie is geen teken dat vibecoding niet werkt. Frustratie is vaak een teken dat je midden in het echte leerproces zit.

GOEDE EERSTE PROJECTEN EN SLECHTE EERSTE PROJECTEN

Het helpt om daar expliciet onderscheid in te maken.

Goede eerste projecten:

- een tooltje dat een terugkerende taak versimpelt
- een eenvoudige persoonlijke site
- een pagina die één dataset of bron bruikbaar maakt
- een minimale interne workflowhulp
- een app met één duidelijke handeling of uitkomst

Slechte eerste projecten:

- een compleet platform met allerlei rollen en rechten
- een product dat meteen voor iedereen moet werken
- iets dat volledig afhankelijk is van meerdere koppelingen
- een project dat tegelijk mooi, slim, schaalbaar en af moet zijn
- een idee dat je zelf nog niet goed genoeg kunt uitleggen

Het verschil zit niet in grootte als statussymbool. Het verschil zit in leerbaarheid.

DE EERSTE DRIE PROJECTEN ZOUDEN ONGEVEER ZO MOETEN VOELEN

Als ik iemand nu vanaf nul aan vibecoding zou laten beginnen, zou ik ongeveer deze route kiezen.

Eerste project: iets voor jezelf. Een klein hulpmiddel. Een simpele tool. Iets waarvan je meteen weet of het werkt.

Tweede project: iets met iets meer vorm en gebruikservaring. Nog steeds beperkt, maar zichtbaarder.

Derde project: iets met meer logica of data, maar alleen als je na de eerste twee snapt waar jouw zwakke plekken zitten.

Wat ik niet zou aanraden: meteen beginnen aan het project waar al je dromen, ambities en ongeduld tegelijk in zitten.

Die mag best bestaan. Maar niet als je eerste oefenterrein.

KLEIN IS OOK EEN MANIER VAN DENKEN

Er zit nog iets onder deze hele aanpak. Klein beginnen is niet alleen een praktische tip. Het is ook een mentale discipline.

Het dwingt je om keuzes te maken.

Wat is nu echt de kern? Wat moet deze eerste versie minimaal kunnen? Wat is luxe? Wat is afleiding? Wat kan later?

Zodra je die vragen goed leert stellen, word je een betere maker. Niet alleen een betere vibecoder.

En dat is uiteindelijk de diepere winst.

DE EERSTE REGEL

Als ik dit hoofdstuk tot één regel moest terugbrengen, zou het deze zijn:

Maak je eerste project zo klein dat je het bijna onder je niveau vindt, maar zo echt dat je er iets van leert.

Dat voelt misschien bescheiden.

In de praktijk is het juist agressief effectief.

HOOFDSTUK 4 - WERK VANUIT TAKEN, NIET VANUIT TOOLS

Een van de snelste manieren om jezelf kwijt te raken in vibecoding is verliefd worden op tools.

Dat klinkt misschien vreemd, want tools zijn juist de zichtbare ingang van deze hele beweging. Het zijn de plekken waar je experimenteert, bouwt, verrast wordt en soms in een paar minuten verder komt dan vroeger in weken. Natuurlijk doen ze ertoe.

Maar precies daarom zijn ze ook gevaarlijk.

Zodra je te veel naar tools kijkt, ga je al snel denken als een verzamelaar in plaats van als een maker. Dan verschuift je aandacht van:

- welk probleem probeer ik op te lossen?
- welke taak wil ik lichter maken?
- welke workflow wil ik verbeteren?

naar:

- welke tool is nu hot?
- welke interface oogt het slimst?
- waar zit de meeste hype?

- wat moet ik eigenlijk ook nog proberen?

Dat is een begrijpelijke reflex.

En toch is het meestal de verkeerde.

Want tools komen en gaan sneller dan taken.

DE VERKEERDE VRAAG

Veel mensen beginnen in vibecoding met een vraag die logisch klinkt, maar hen meteen op het verkeerde spoor zet:

welke tool moet ik gebruiken?

Dat lijkt praktisch. In werkelijkheid is het vaak te vroeg.

De betere eerste vraag is:

wat probeer ik precies gedaan te krijgen?

Wil je een eerste prototype maken? Een tekst herschrijven? Een kleine workflow automatiseren? Een dataset bruikbaar maken? Een site publiceren? Een intern hulpmiddel bouwen?

Zodra je dat weet, wordt de keuze voor een tool kleiner, rustiger en minder ideologisch.

Wie begint bij tools, denkt vaak in merken. Wie begint bij taken, denkt in werk.

En werk is uiteindelijk waar het om gaat.

TAKEN BLIJVEN, INTERFACES SCHUIVEN

Een van de structurele misverstanden in AI-land is dat mensen doen alsof elk nieuw product een compleet nieuw tijdperk opent.

Soms gebeurt dat ook echt. Vaak verschuift er vooral iets in de interface.

De onderliggende taken blijven opvallend stabiel.

Je wilt nog steeds:

- iets uitzoeken
- iets ordenen
- iets schrijven
- iets samenvatten
- iets publiceren
- iets bouwen
- iets terugvinden
- iets terugkerends vereenvoudigen

Wat verandert, is hoe direct, hoe snel en hoe zelfstandig je dat kunt doen.

Vandaag gebruik je daar misschien een specifieke tool voor. Morgen zit die mogelijkheid in een andere omgeving, in een model zelf of in een productlaag die er nu nog niet is. Als je jouw manier van werken te strak ophangt aan één interface, moet je steeds opnieuw beginnen zodra de markt verschuift.

Als je werkt vanuit taken, blijft je werkwijze veel beter overeind.

EEN MAKER ORGANISEERT GEEN TOOLSET, MAAR EEN WERKSTROOM

Dat is voor mij een belangrijk onderscheid.

Veel mensen zijn bezig een collectie tools op te bouwen. Ze hebben favorieten, screenshots, lijstjes, bookmarks en accountjes overal.

Maar een maker die duurzaam wil werken, heeft iets anders nodig.
Niet alleen gereedschap. Een werkstroom.

Dat betekent:

- weten welke soorten taken je vaak hebt
- weten in welke volgorde je ze meestal doet
- weten waar je ruwe input landt
- weten hoe je tussenresultaten bewaart
- weten wanneer iets nog experiment is en wanneer iets bron wordt

Dat klinkt minder sexy dan een nieuwe tool uitproberen. Het is ook veel waardevoller.

Zodra je werkstroom helder is, kun je veel makkelijker wisselen van hulpmiddel zonder je hele praktijk te verliezen.

TOOLONAFHANKELIJK WERKEN IS GEEN PURISME

Soms klinkt onafhankelijk willen werken alsof je principieel tegen platforms of grote aanbieders bent. Zo bedoel ik het niet.

Je hoeft niet krampachtig alles zelf te hosten, elk systeem te wantrouwen of je werk op te sluiten in ingewikkelde eigen constructies.

Onafhankelijkheid is hier iets praktischers.

Het betekent:

- dat je begrijpt waar je werk staat
- dat je weet wat de bron van waarheid is
- dat je output kunt meenemen als een tool verdwijnt

- dat je niet voor elke kleine stap opnieuw hoeft te leren denken

Dat is geen ideologische luxe. Dat is gewoon verstandig werken.

In een speelveld dat zo snel beweegt als dit, is lock-in niet alleen een technisch risico. Het is ook een mentaal risico. Je gaat dan namelijk onbewust denken binnen de grenzen van een bepaald product, terwijl jouw werkvraag vaak breder is.

DE LAAG BOVEN DE MODELLEN WORDT BELANGRIJK

Er is nog iets anders aan de hand. Veel vibecodingtools voelen nu vernieuwend omdat ze een prettige ingang bieden tot krachtige modellen. Maar die verhouding schuift voortdurend.

Soms loopt een gespecialiseerde tool voorop. Soms halen modelmakers functies naar zich toe. Soms worden verschillen kleiner. Soms wint de tool niet op pure intelligentie, maar op workflow, context, geheugen, publicatiegemak of samenwerking.

Dat is precies waarom je niet moet doen alsof één tool het hele antwoord is.

De echte vraag is vaak:

welke laag voegt voor mijn werk iets wezenlijks toe?

Bijvoorbeeld:

- een goede omgeving om ideeën te verkennen
- een plek om langere context vast te houden
- een manier om bronmateriaal gestructureerd te bewaren
- een snelle route naar publicatie
- een handige bouwomgeving voor een eerste prototype

Dat zijn werkvragen, geen fanvragen.

EEN SIMPEL BESLISKADER

Je hebt in de praktijk geen grote theorie nodig om toolrustiger te worden. Een eenvoudig besliskader helpt vaak al.

Vraag bij elke taak:

1. Wat voor soort werk is dit?
2. Heb ik vooral denken, schrijven, structureren, automatiseren of bouwen nodig?
3. Heb ik veel context nodig of juist snelheid?
4. Is dit tijdelijk experiment of blijvende output?
5. Moet ik dit later makkelijk kunnen meenemen naar een andere omgeving?

Pas daarna komt de toolkeuze.

Zo'n kader haalt de emotie uit de beslissing. Je kiest dan niet meer vooral op nieuwigheid, merkgevoel of online enthousiasme, maar op geschiktheid.

Dat scheelt ruis.

VOORBEELDEN VAN TAAKDENKEN

Neem drie totaal verschillende situaties.

Je wilt een eerste concepttekst maken. Dan heb je vooral iets nodig dat goed meedenkt, structuur aankan en snel varianten levert.

Je wilt een kleine werkapp bouwen. Dan heb je vooral iets nodig dat een interface kan opzetten, logica kan uitwerken en snel iteraties mogelijk maakt.

Je wilt een terugkerende handmatige taak lichter maken. Dan heb je misschien helemaal geen “vibecodingplatform” nodig, maar een eenvoudige automation, een script of een lichte tool die één stap uit handen neemt.

In alle drie gevallen kun je eindeloos discussiëren over welke tool het best is. Maar de echte winst zit eerder in het goed begrijpen van de taak zelf.

HYPE MAAKT DOMMER DAN NODIG IS

Er is nog een reden om taakgericht te blijven: hype vernauwt je blik.

Zodra één tool dominant in de aandacht komt, gaan mensen die tool op alles proberen te plakken. Niet omdat dat verstandig is, maar omdat het mentaal makkelijk is. Je hebt iets nieuws ontdekt en wilt dat overal op toepassen.

Daardoor krijg je twee problemen.

Het eerste is overschatting. Je verwacht te veel van één omgeving.

Het tweede is luiheid. Je stopt met goed nadenken over de aard van het werk.

Dan ga je niet meer vragen:

- is dit eigenlijk een schrijfprobleem?
- een ordeningsprobleem?
- een bouwprobleem?
- een publicatieprobleem?
- een archiveringsprobleem?

En juist dat onderscheid maakt je beter.

JE EIGEN WERKWIJZE IS EEN ASSET

Een goede vibecoder heeft meer dan een voorkeur voor bepaalde tools. Hij bouwt een eigen manier van werken.

Je eigen werkwijze bestaat bijvoorbeeld uit:

- hoe je ideeën opvangt
- hoe je beslist wat de moeite waard is
- hoe je bronmateriaal bewaart
- hoe je van losse input naar concept gaat
- hoe je prototypes test
- hoe je bepaalt wat blijvend is en wat weg mag

Zodra die laag stevig wordt, kun je allerlei technologische verschuivingen beter verdragen. Dan verlies je niet telkens je grip als een tool verandert, duurder wordt, slechter wordt of verdwijnt.

Dan houd je namelijk iets belangrijkers over:

een eigen maakpraktijk.

PORTABILITEIT IS RUST

Veel mensen onderschatten hoe prettig het is als je werk verplaatsbaar blijft.

Niet alleen bestanden. Ook denkwijzen, workflows en tussenlagen.

Als je notities, teksten, bronnen, bouwstenen en prototypes allemaal opgesloten zitten in één omgeving, voelt dat vaak eerst comfortabel. Alles zit bij elkaar. Alles lijkt handig. Tot het moment komt waarop je weg wilt, moet opschalen of simpelweg merkt dat die omgeving niet meer past.

Dan blijkt gemak opeens afhankelijkheid te zijn geworden.

Portabiliteit is daarom niet alleen een technisch woord. Het is ook mentale rust. Je weet dat je kunt verschuiven zonder jezelf kwijt te raken.

WERK LIEVER VANUIT ECHTE VRAGEN

Dit is misschien mijn belangrijkste voorkeur in dit hoofdstuk.

Ik geloof niet zo in bouwen vanuit abstracte fascinatie voor tools alleen. Dat levert soms leuke experimenten op, maar zelden een duurzame praktijk.

De beste vibecodingprojecten ontstaan meestal uit echte vragen:

- iets dat je zelf steeds opnieuw moet doen
- iets dat je irriteert
- iets dat nergens goed genoeg voor bestaat
- iets dat je voor een kleine groep bruikbaar wilt maken

Daar zit frictie, context en urgentie in. Daardoor worden keuzes meestal beter. Je pakt dan niet zomaar een tool omdat iedereen hem noemt, maar iets dat helpt bij een concreet stuk werk.

DE VOLWASSEN VRAAG

Uiteindelijk komt dit hoofdstuk uit bij een simpele vraag:

welke combinatie van werkwijze, opslag, context en hulpmiddelen helpt mij dit werk goed doen?

Het is een minder sexy vraag dan wat nu de slimste tool is, maar wel de vraag van iemand die een praktijk opbouwt in plaats van een verzameling favorieten.

HOOFDSTUK 5 - VAN IDEE NAAR EERSTE BRUIKBARE VERSIE

De grootste kracht van vibecoding zit niet in snelheid op zichzelf. Snelheid zonder richting levert vooral meer rommel op.

De echte winst zit in het kleiner maken van de afstand tussen een vaag idee en een eerste versie waar je iets mee kunt. Geen eindproduct. Geen glanzende demo. Gewoon iets dat genoeg vorm heeft om te testen, te beoordelen en desnoods weg te gooien.

Veel ideeën sneuvelen doordat ze te lang vaag blijven. Ze blijven hangen in notities, screenshots, losse ingevoingen en zinnen als: hier moet ik echt eens iets mee.

Vibecoding kan dat patroon doorbreken, mits je tempo koppelt aan richting.

EEN IDEE IS NOG GEEN OPDRACHT

Bijna elk project begint met een gevoel, irritatie of ingeving.

Je denkt:

- waarom bestaat hier geen simpele oplossing voor?
- waarom kost dit me elke week zoveel tijd?
- waarom moet ik dit steeds handmatig doen?
- waarom voelt bestaande software alsof die net niet voor mij gemaakt is?

Dat zijn goede beginsignalen. Maar het zijn nog geen bruikbare opdrachten.

Een van de grootste verschillen tussen een idee dat blijft zweven en een idee dat beweegt, is of je het scherp genoeg kunt maken voor een eerste bouwronde.

Niet:

"maak een geweldige app voor creatieve zelfstandigen"

Wel:

"maak een eenvoudige webpagina waarop ik links kan bewaren, taggen en terugvinden, zodat ik mijn research voor één project overzichtelijker kan houden"

Dat verschil is alles.

Een bruikbare eerste opdracht bevat meestal drie dingen:

- voor wie het is
- welk probleem je oplost
- wat de eerste versie minimaal moet kunnen

Meer heb je in het begin vaak niet nodig.

DE EERSTE VERSIE IS GEEN VISITEKAARTJE

Een fout die veel mensen maken, is dat ze hun eerste versie behandelen alsof die meteen toonbaar, netjes en afgerond moet zijn. Daardoor ontstaat er vroeg in het proces te veel druk.

De eerste versie heeft een andere taak.

Die moet niet imponeren. Die moet onthullen.

Een eerste versie laat zien:

- of je het probleem goed ziet
- of de gekozen vorm logisch is
- of de kernhandeling werkt
- of je eigenlijk iets anders bedoelde dan je eerst dacht

Dat betekent ook dat de eerste versie niet per se mooi hoeft te zijn. Niet compleet. Niet schaalbaar. Niet toekomstbestendig.

Als hij maar genoeg doet om jou iets te leren.

Wie een eerste versie probeert te behandelen als bijna-eindproduct, maakt het proces vaak zwaarder dan nodig. Je gaat dan al vroeg sleutelen aan bijzaken: kleuren, kopjes, animaties, accountstructuren, exportfuncties, rechten, koppelingen en opsmuk. Terwijl de echte vraag nog veel eenvoudiger is:

werkt de kern eigenlijk?

DE STANDAARD WERKLOOP

Voor vibecoding werkt een eenvoudige werkwloop meestal beter dan een groots meesterplan.

Voor mij ziet die lus er ongeveer zo uit:

1. benoem het probleem zo concreet mogelijk
2. kies de kleinste bruikbare versie
3. laat een eerste versie bouwen
4. test alleen de kern
5. beslis bewust: doorbouwen, terugdraaien of opnieuw beginnen

Meer is het niet.

Maar elk van die stappen verdient discipline.

STAP 1: BENOEM HET PROBLEEM ZO CONCREET MOGELIJK

Veel mensen beginnen met de oplossing. Dat is verleidelijk, maar meestal te vroeg.

Ze zeggen:

- ik wil een app
- ik wil een dashboard
- ik wil een AI-assistent
- ik wil een platform

Maar vaak weet je dan nog niet scherp genoeg welk probleem dat ding precies moet oplossen.

Begin liever hier:

- wat gaat er nu mis?
- wat kost me nu tijd, aandacht of energie?
- wat doe ik nu handmatig dat eigenlijk eenvoudiger kan?
- welke informatie of handeling wil ik sneller, rustiger of helderder maken?

Zodra je dat weet, wordt de rest eenvoudiger.

Een goede eerste projectomschrijving klinkt vaak bijna saai. Dat is een goed teken. Saaie helderheid wint het meestal van ambitieuze vaagheid.

STAP 2: KIES DE KLEINSTE BRUIKBARE VERSIE

Dit is het moment waarop je streng moet zijn.

Niet: wat zou er uiteindelijk allemaal mooi zijn?

Wel: wat moet deze eerste versie minimaal doen om nuttig te zijn?

Bijvoorbeeld:

- één formulier in plaats van een volledig systeem

- één pagina in plaats van een heel platform
- één workflow in plaats van een complete suite
- één type gebruiker in plaats van meerdere rollen

Dat is geen versmalling uit angst. Dat is versmalling uit intelligentie.

Zodra je te veel tegelijk wilt, weet je niet meer wat je aan het testen bent. Zodra je klein genoeg kiest, wordt het zichtbaar of de kern hout snijdt.

STAP 3: LAAT EEN EERSTE VERSIE BOUWEN

Nu pas komt de daadwerkelijke vibecoding in beeld.

Ook hier helpt het om niet te breed te praten. Een goede eerste instructie is meestal niet lang, maar helder.

Zoiets als:

"Bouw een eenvoudige, rustige webapp voor mijn eigen research. Ik wil links kunnen toevoegen met titel, bron en korte notitie. Toon de links in een overzichtelijke lijst. De eerste versie hoeft geen login of koppelingen te hebben. Het doel is dat ik sneller eerdere bronnen terugvind."

Wat hier goed aan is:

- de gebruiker is duidelijk
- het doel is duidelijk
- de kernfuncties zijn duidelijk
- wat nog niet hoeft, is ook duidelijk

Dat laatste wordt vaak vergeten.

Een goede opdracht zegt niet alleen wat er wel moet komen, maar ook wat er nu nog buiten scope valt. Daarmee voorkom je dat een

tool alvast allerlei extra's gaat verzinnen waar jij nog niets aan hebt.

STAP 4: TEST ALLEEN DE KERN

Als de eerste versie er is, willen veel mensen meteen alles beoordelen.

Ze kijken tegelijk naar:

- de vormgeving
- de snelheid
- de details
- de mobiele weergave
- foutmeldingen
- extra functies
- slimme uitbreidingen

Dat is te veel ineens.

Test eerst de kernhandeling.

Bij een researchtool is dat misschien: kan ik eenvoudig iets toevoegen en later terugvinden?

Bij een kleine site: is meteen duidelijk wat dit is en wat ik hier kan doen?

Bij een workflowhulp: maakt dit de taak echt lichter of alleen maar anders?

De eerste testvraag is zelden: is dit al goed genoeg voor de buitenwereld?

De eerste testvraag is: doet dit iets wezenlijks beter dan niets?

STAP 5: KIES BEWUST WAT JE NU DOET

Dit is misschien wel de belangrijkste stap, omdat hier veel makers ongemerkt tijd verliezen.

Na een eerste versie zijn er meestal drie routes.

1. Doorbouwen Als de kern klopt en je ziet wat nog beter moet, bouw je gericht verder.
1. Terugdraaien Als de richting goed voelt maar een recente wijziging het slechter maakte, ga je een stap terug.
1. Opnieuw beginnen Als de basis niet goed is, de structuur rommelig is of de tool je op een onhandig spoor heeft gezet, is opnieuw beginnen vaak slimmer dan eindeloos repareren.

Dat laatste vinden veel mensen lastig, omdat opnieuw beginnen voelt als verlies.

Vaak is het juist winst.

Een mislukte eerste versie is zelden weggegooid tijd als je nu veel scherper weet:

- wat de kern is
- wat overbodig bleek
- wat je anders moet vragen
- welke keuzes je niet opnieuw wilt maken

Soms moet het wiel niet eindeloos gerepareerd worden. Soms moet het opnieuw uitgevonden blijven worden, maar dan beter.

WANNEER JE MOET DOORBOUWEN

Doorbouwen is verstandig als:

- de kernhandeling werkt
- het probleem goed gekozen voelt
- de structuur nog te redden is
- de verbeteringen duidelijk zijn

Dan heeft het zin om in kleine slagen te verfijnen.

Niet alles tegelijk. Steeds een beperkte verbetering:

- maak het overzicht rustiger
- voeg zoeken toe
- verbeter de invoer
- maak de lege staat duidelijker
- herstel één fout in de logica

Zo blijft het project bestuurbaar.

WANNEER JE BETER OPNIEUW KUNT BEGINNEN

Opnieuw beginnen is vaak slimmer als:

- de eerste versie een warboel is geworden
- je oorspronkelijke vraag eigenlijk verkeerd was
- de tool veel te veel heeft gebouwd
- de basislogica niet klopt
- je tijdens het maken ontdekt dat het echte probleem ergens anders zit

Veel beginnende vibecoders blijven dan te lang pielen. Ze hopen dat nog een prompt, nog een kleine fix of nog een extra laag het project alsnog redt.

Soms gebeurt dat. Vaak niet.

HOOFDSTUK 6 - SMAAK, OORDEEL EN PRODUCTGEVOEL

Een van de hardnekkigste misverstanden over vibecoding is dat mensen denken dat de belangrijkste drempel nu weg is.

Dat is maar half waar.

Ja, de technische drempel is lager geworden. Veel meer mensen kunnen iets laten ontstaan dan een paar jaar geleden. Dat is echt. Maar zodra die eerste barriere lager wordt, verschijnt er meteen een andere vraag op de voorgrond:

is het ook goed?

Niet: werkt het een beetje? Niet: ziet het er indrukwekkend uit in een demo? Niet: heeft het genoeg functies?

Maar:

is het helder? is het prettig? is het logisch? is het bruikbaar? is het precies genoeg?

Daar komen smaak, oordeel en productgevoel binnen.

En dat zijn geen luxe extra's. Dat zijn precies de dingen die belangrijker worden zodra meer mensen kunnen bouwen.

DE TOOL MAAKT GEEN GOED PRODUCT

Dit is misschien de eerlijkste regel van het hele boek:

een sterke tool garandeert nog geen sterk product.

Een tool kan snel code genereren. Een interface opzetten. Een stijl voorstellen. Een tekst schrijven. Een workflow nabootsen.

Maar hij weet niet vanzelf wat goed is voor jouw gebruiker, jouw context en jouw doel.

Hij weet niet automatisch:

- wat weg kan
- wat verwarrend is
- wat te veel is
- wat net niet klopt
- waar de aandacht naartoe moet
- wat rust geeft
- wat vertrouwen opwekt

Dat moet nog steeds ergens vandaan komen.

En meestal komt dat van de maker.

SMAAK IS NIET HETZELFDE ALS DECORATIE

Als mensen het over smaak hebben, denken ze vaak eerst aan visuele stijl. Mooie kleuren. Goede typografie. Nettere knoppen. Dat hoort erbij, maar het is kleiner dan ik hier bedoel.

Smaak in software is ook:

- weten wat je weglaat
- aanvoelen wat te druk is
- herkennen wat de kernhandeling overschaduwt
- voelen wanneer een interface schreeuwerig wordt
- snappen wat een gebruiker op dit moment nodig heeft en wat niet

Smaak is dus niet alleen een kwestie van er iets goed uit laten zien. Smaak is ook een manier van beslissen.

Een product met smaak voelt vaak rustiger dan een product zonder smaak. Niet omdat het minder kan, maar omdat het minder tegelijk wil zijn.

MEER OUTPUT MAAKT OORDEEL SCHAARSER

Dat is een van de grote verschuivingen van dit moment.

Als productie makkelijker wordt, krijg je meer output. Meer prototypes. Meer apps. Meer sites. Meer tools. Meer half-afgewerkte ideeën die in recordtijd iets zichtbaars zijn geworden.

Dat is opwindend. Maar het maakt ook iets anders schaarser.

Niet productie, maar selectie.

Niet genereren, maar beoordelen.

Niet maken, maar kiezen.

Want als je tien versies kunt laten ontstaan in de tijd waarin je vroeger misschien één eerste schets had, dan wordt de kwaliteit van jouw oordeel ineens veel bepalender.

Welke versie werkt beter? Welke richting voelt helderder? Welke toevoeging helpt echt? Welke verbetering is eigenlijk alleen maar meer?

Dat zijn productvragen.

PRODUCTGEVOEL IS ZIEN WAT ER MOET GEBEUREN

Productgevoel klinkt soms vaag, maar in de praktijk is het juist heel concreet.

Het is het vermogen om te zien:

- wat hier de hoofdhandeling is
- waar frictie zit
- wat overbodig is
- wat nog ontbreekt
- welke keuze de ervaring simpeler maakt
- wanneer een oplossing slimmer lijkt dan hij is

Mensen met goed productgevoel zijn niet altijd de luidruchtigste mensen in een proces. Vaak zijn het juist de mensen die iets kunnen terugbrengen tot de essentie.

Ze zien eerder:

- deze knop hoeft hier niet
- deze tekst is te lang
- deze stap maakt het onduidelijker
- deze functie is interessant maar nog niet nodig
- deze pagina wil drie dingen tegelijk

Dat is geen technische superioriteit. Dat is richtinggevoel.

DE GEBRUIKER ZIET GEEN PROMPT

Dit is een simpele maar belangrijke realiteit.

De gebruiker ziet jouw prompt niet. Die ziet het resultaat.

Het maakt voor een gebruiker niet uit of iets is gebouwd door een klassiek developmentteam, een solo maker met AI-hulp of een mix van beide. Wat telt is:

- snap ik het?
- vertrouw ik het?
- helpt dit me?

- kost dit me moeite?
- voelt dit af?

Dat betekent ook dat vibecoding je niet ontslaat van zorgvuldigheid. Misschien juist minder dan ooit.

Als bouwen sneller wordt, is het verleidelijk om te denken dat rommeligheid ook wel hoort bij de nieuwe tijd. Soms is dat even prima in een experiment. Maar zodra iets voor andere mensen bedoeld is, gelden nog steeds steeds dezelfde oude wetten van duidelijkheid, vertrouwen en bruikbaarheid.

GOED VRAGEN STELLEN IS CREATIEF WERK

Een onderschat punt: smaak en productgevoel zitten niet pas aan het einde van het proces, als je iets moet beoordelen.

Ze zitten al aan het begin, in wat je vraagt.

De maker met smaak vraagt anders.

Niet alleen:

"maak een moderne app"

Maar eerder:

"maak een rustige interface met een duidelijke hoofdroutine, veel witruimte, weinig afleiding en een eerste scherm waarop meteen duidelijk is wat de gebruiker hier kan doen"

Of:

"bouw dit zo dat de belangrijkste handeling centraal staat en secundaire opties pas later zichtbaar worden"

Dat is meer dan prompttechniek. Dat is een productvisie in compacte vorm.

Hoe beter jij voelt wat een goede ervaring is, hoe scherper je kunt sturen tijdens het maken.

MINDER, MAAR BETER, WINT VAAK

Een van de gevaren van vibecoding is overvloed.

Omdat toevoegen zo makkelijk wordt, wordt weglaten moeilijker.

Je kunt nog een functie vragen. Nog een variant. Nog een blok. Nog een menu. Nog een grafiek. Nog een slimme stap.

En voor je het weet, is iets dat begon als een heldere oplossing veranderd in een druk geheel dat van alles kan, behalve goed helpen.

Daarom is een van de belangrijkste productregels ook hier weer verrassend ouderwets:

minder, maar beter.

Een goede eerste tool hoeft niet rijk te voelen. Hij moet raak voelen.

Een goede pagina hoeft niet veel te zeggen. Hij moet duidelijk zeggen wat nodig is.

Een goede workflow hoeft niet indrukwekkend te zijn. Hij moet soepel zijn.

GOEDE SMAAK IS VAAK ZICHTBAAR IN WAT ONTBREEKT

Bij zwakke producten zie je vaak alles wat erin is gestopt. Bij sterke producten voel je vooral wat er bewust uit is gebleven.

Dat is een belangrijk onderscheid.

Goede smaak laat zich niet alleen zien in wat aanwezig is. Ook in wat is tegengehouden.

Niet elke mogelijkheid hoeft meteen zichtbaar te zijn. Niet elke gedachte hoeft een functie te worden. Niet elke technische optie hoeft een keuze voor de gebruiker te worden.

Dat is soms moeilijk, zeker als je zelf maker bent en weet hoeveel er mogelijk is. Maar voor gebruikers is jouw interne opwinding zelden de beste gids.

Hun ervaring is de maatstaf.

EEN KLEINE CHECKLIST VOOR BRUIKBARE INTERFACES

Je hoeft geen opgeleid ontwerper te zijn om een eerste kwaliteitscheck te doen.

Deze vragen helpen bijna altijd:

- is meteen duidelijk wat dit is?
- is meteen duidelijk wat ik hier kan doen?
- wat is de belangrijkste handeling op dit scherm?
- staat er iets in de weg?
- is dit drukker dan nodig?
- vraag ik hier te veel aandacht van de gebruiker?
- voelt dit als een hulpmiddel of als een hindernis?

Dat zijn simpele vragen. Maar ze brengen je vaak sneller bij kwaliteit dan een lang debat over stijltrends.

SMAAK GROEIT DOOR KIJKEN EN VERGELIJKEN

Het lastige van smaak is dat je het niet zomaar uit een model kunt downloaden.

Je ontwikkelt het door:

- goed te kijken
- veel te vergelijken
- irritatie serieus te nemen
- te merken wat jou rust geeft
- te herkennen wat vertrouwen schept
- terug te zien waarom iets in de praktijk prettig werkt

Dat geldt voor schrijven, vormgeving, muziek, film en ook voor software.

Wie alleen leert hoe hij output moet genereren, wordt misschien productief. Wie ook leert zien waarom iets goed of slecht aanvoelt, wordt gevaarlijker op een goede manier.

HET NIEUWE VERSCHIL ZIT NIET ALLEEN IN KUNNEN BOUWEN

Naarmate vibecoding normaler wordt, verschuift het onderscheid tussen makers.

HOOFDSTUK 7 - HET SPEELVELD VERANDERT SNEL, DUS JOUW HOUDING MOET MEEBEWEGEN

Als je te lang in de AI-hoek rondloopt, raak je makkelijk verblind door snelheid.

Elke week is er wel iets nieuws. Een model dat beter redeneert. Een tool die meer kan bouwen. Een interface die toegankelijker

voelt. Een product dat functies opslokt die vorige maand nog onderscheidend leken.

Dat heeft iets opwindends. Maar het heeft ook een nadeel: het maakt mensen onrustig.

Ze krijgen het gevoel dat ze continu moeten bijhouden wat er verandert, anders raken ze achter. En dus springen ze van update naar update, van tool naar tool, van belofte naar belofte.

Dat is begrijpelijk. En toch is het geen volwassen houding.

Want als het speelveld snel beweegt, heb je niet alleen nieuwsgierigheid nodig. Je hebt vooral een manier nodig om niet bij elke verschuiving opnieuw jezelf kwijt te raken.

DE NAÏEVE FASE IS VOORBIJ

In de eerste golf van vibecoding zat veel verwondering.

Terecht ook. Voor veel mensen was het al spectaculair dat je in gewone taal een bruikbare interface of simpele tool kon laten ontstaan. Die fase had iets pionierachtigs. Alles voelde nieuw. Alles voelde mogelijk.

Maar na zo'n eerste golf komt altijd een tweede fase.

Dan wordt het speelveld serieuzer. Niet minder interessant, maar wel minder onschuldig.

De eerste wow-momenten zijn dan niet meer genoeg. Je merkt dat:

- veel tools op elkaar gaan lijken
- modelmakers functies zelf naar zich toetrekken
- gespecialiseerde producten hun voorsprong moeten bewijzen
- gebruikers hogere eisen gaan stellen
- eigenaarschap en afhankelijkheid zwaarder gaan wegen

Dat is een teken van volwassenwording.

Niet van het einde van de beweging. Wel van het einde van de simpelste fase.

MODELMAKERS EN TOOLMAKERS SCHUIVEN NAAR ELKAAR TOE

Een van de belangrijkste verschuivingen van nu is dat de grens tussen model en productlaag minder stabiel wordt.

Soms wordt een gespecialiseerde tool groot doordat hij een slimmere werkstroom rond een model bouwt. Soms voegt een modelmaker zelf juist functies toe die die extra laag minder uniek maken. Soms ontstaan er nieuwe combinaties waarin het onderscheid tussen chat, builder, editor en automation vervaagt.

Voor gebruikers betekent dat iets heel praktisch:

je moet niet verliefd worden op tijdelijke grenzen.

Wat vandaag voelt als een aparte categorie, kan morgen ingebakken zijn in iets anders. Wat nu een premium onderscheid is, kan over een paar maanden basisfunctionaliteit zijn.

Dat is geen reden voor cynisme. Wel voor nuchterheid.

TOEGANKELIJKHEID WORDT NORMALER

Nog een structurele lijn: de interfaces worden eenvoudiger.

Wat eerst voelde als iets voor mensen die veel geduld, veel experimenteerdrift of een redelijke technische tolerantie hadden, schuift steeds meer richting bredere werkpraktijk.

Dat gebeurt op verschillende manieren:

- betere interfaces

- meer begeleiding in het proces
- toegankelijker publicatieroutes
- minder technische drempels in setup en deployment
- meer ingebouwde hulp bij de eerste stappen

Dat is belangrijk, want het betekent dat vibecoding langzaam uit de speeltuin komt.

Niet alles blijft hobby, demo of ontdekking. Meer en meer komt het terecht in echt werk.

In kleine teams. Bij zelfstandigen. In redacties. In onderwijs. In interne workflows. In nicheprojecten.

Dat maakt de kwaliteit van je houding nog belangrijker.

NIEUWSGIERIGHEID ALLEEN IS NIET GENOEG

In de beginfase van een nieuwe beweging kun je ver komen op enthousiasme.

Je probeert dingen. Je klikt rond. Je test van alles. Je kijkt wat blijft hangen.

Dat is gezond. Maar op een gegeven moment wordt nieuwsgierigheid zonder selectie vermoeiend.

Dan heb je iets anders nodig:

- criteria
- voorkeuren
- werkprincipes
- een eigen grens tussen signaal en ruis

Anders blijf je permanent in de verkenningstand staan.

En verkenning is geen praktijk.

WAT WAARSCHIJNLIJK STRUCTUREEL IS

Niet alles in dit speelveld blijft zoals het nu is. Maar een paar dingen lijken me wel degelijk structureel.

Ten eerste: de afstand tussen idee en eerste uitvoering blijft kleiner dan vroeger.

Ten tweede: meer niet-traditionele makers zullen software als werkvorm blijven gebruiken.

Ten derde: de bottleneck blijft verschuiven van pure uitvoering naar richting, selectie en kwaliteit.

Ten vierde: kleine, specifieke en persoonlijke software blijft interessanter dan voorheen.

Ten vijfde: de laag boven de modellen, dus workflow, context, publicatie, samenwerking en opslag, blijft belangrijk, ook als de modellen zelf steeds beter worden.

Dat zijn geen details. Dat zijn de lijnen waar je je werkwijze op kunt bouwen zonder dat je bij elke update opnieuw hoeft te beginnen.

WAT WAARSCHIJNLIJK TIJDELIJK IS

Er is ook genoeg dat je niet te serieus moet nemen.

Bijvoorbeeld:

- de absolute dominantie van één tool
- stellige claims dat nu alles anders is geworden
- marketingtaal die elke functie als revolutie verkoopt
- modieuze termen die meer verwarring dan helderheid brengen
- het idee dat één setup je definitief toekomstbestendig maakt

Dat soort dingen hoort bij een jong speelveld. Je hoeft ze niet te negeren. Maar je moet ze ook niet verwarren met de diepere beweging.

EIGENAARSCHAP WORDT BELANGRIJKER

Naarmate vibecoding serieuzer wordt, verschuift ook de vraag van: kan ik dit maken?

naar:

van wie is dit eigenlijk?

Waar staat je werk? Waar staat je context? Waar staat je bronmateriaal? Wat gebeurt er als een tool verdwijnt? Wat neem je mee? Wat verlies je?

Dat zijn geen randvragen meer. Zeker niet als je bouwt aan iets dat verder moet leven dan een test of demo.

De volwassen maker denkt daarom niet alleen aan output, maar ook aan eigenaarschap.

Niet uit paranoia. Uit verantwoordelijkheid.

VAN SPEELTUIN NAAR WERKVLOER

Dit is misschien wel de belangrijkste verschuiving van de komende tijd.

Vibecoding blijft natuurlijk ook een plek voor spel, experiment en toeval. Dat is goed. Maar steeds vaker zal het gebruikt worden voor werk dat echt ergens toe doet.

Dan worden andere dingen relevant:

- betrouwbaarheid

- overdraagbaarheid
- onderhoud
- publicatie
- archivering
- samenwerking

Met andere woorden: je kunt niet eeuwig blijven doen alsof alles een proefballon is.

Zodra iets echte waarde krijgt in je werk, moet je ook volwassenere eisen aan jezelf stellen.

JOUW HOUDING IS HET STABIELSTE ONDERDEEL

Dat is uiteindelijk de reden voor dit hoofdstuk.

In een snel verschuivend speelveld zijn tools niet het stabielste element. Interfaces niet. Merken niet. Modellen niet.

Jouw houding is stabiel.

Daarmee bedoel ik:

- hoe je kiest
- hoe je test
- hoe je opslaat
- hoe je beoordeelt
- hoe je voorkomt dat je afhankelijk wordt zonder het te merken
- hoe je experimenteert zonder je focus te verliezen

Wie die laag op orde krijgt, kan veel beter meebewegen zonder stuurloos te worden.

EEN PAAR HOUDINGSREGELS DIE HELPEN

Als ik dit hoofdstuk tot een klein stel regels zou terugbrengen, zouden het ongeveer deze zijn:

- verwonder je, maar organiseer je werk niet op verwondering alleen
- test nieuwe dingen, maar verander je hele praktijk niet bij elke update
- bouw je werkwijze op structurele taken, niet op tijdelijke toolvoordelen
- let op eigenaarschap zodra iets meer wordt dan experiment
- houd nieuwsgierigheid, maar koppel haar aan selectie

Dat klinkt misschien minder opwindend dan de nieuwste demo. Dat is precies de bedoeling.

Rust is hier een concurrentievoordeel.

WAT MOET BLIJVEN HANGEN

Als je na dit hoofdstuk maar één ding onthoudt, laat het dan dit zijn:

Het speelveld verandert snel, maar jouw reactie daarop hoeft niet nerveus te zijn.

Je hebt geen eindeloze jacht op updates nodig. Je hebt een betere houding nodig.

Nieuwsgierig genoeg om mee te bewegen. Stevig genoeg om niet bij elke beweging je eigen praktijk kwijt te raken.

HOOFDSTUK 8 - WAT BLIJFT ALS DE

HYPE WEG IS

Elke nieuwe technologiegolf maakt eerst veel lawaai. Nieuwe termen, nieuwe interfaces, nieuwe beloftes, nieuwe voorbeelden van wat nu opeens mogelijk lijkt.

Dat lawaai is niet nutteloos. Het trekt aandacht, opent verbeelding en helpt mensen over een drempel. Maar als je er te lang in blijft hangen, ga je de buitenkant verwarren met de kern.

Daarom wil ik aan het einde van dit boek rustiger kijken. Wat blijft overeind als de hype afneemt, de toolnamen verschuiven en de eerste wow-momenten normaler worden?

DE GEREEDSCHAPPEN VERANDEREN, DE IMPULS BLIJFT

De tools van vandaag zullen niet allemaal de tools van morgen zijn.

Sommige verdwijnen. Sommige worden opgeslokt. Sommige worden ingebakken in andere producten. Sommige verliezen hun glans zodra de rest ze heeft ingehaald.

Dat is niet vreemd. Zo gedragen jonge gereedschappen zich.

Maar onder die verschuivende laag zit iets dat dieper ligt.

Mensen willen maken. Mensen willen hun werk beter laten passen bij hoe zij denken. Mensen willen iets bouwen dat niet bestaat maar wel zou moeten bestaan. Mensen willen minder afhankelijk zijn van systemen die bijna goed zijn, maar net niet.

Die impuls verdwijnt niet als een merknaam minder relevant wordt.

NIET IEDEREEN WORDT PROGRAMMEUR, MAAR VEEL MEER MENSEN WORDEN MAKER

Voor mij is dit nog steeds de hoofdzin van het hele onderwerp.

Het punt is niet dat iedereen nu software-engineer moet worden.

Het punt is dat veel meer mensen de stap kunnen zetten van:

ik heb een idee

naar:

ik kan er iets van laten ontstaan

Dat verandert hoe mensen naar software kijken. Niet meer alleen als iets dat je koopt, gebruikt of aanvraagt. Maar ook als iets dat je kunt vormen.

Dat is een culturele verschuiving die niet zomaar meer teruggaat.

KLEINE SOFTWARE WORDT BLIJVEND INTERESSANTER

Een andere lijn die volgens mij blijft: kleine, gerichte en persoonlijke software wordt structureel aantrekkelijker.

Niet alles hoeft een platform te zijn. Niet alles hoeft honderd functies te hebben. Niet alles hoeft bedoeld te zijn voor miljoenen gebruikers.

Sommige van de waardevolste dingen zullen juist klein zijn:

- een tool voor je eigen werk
- een oplossing voor een kleine community
- een tijdelijke site rond een project
- een interne workflowhulp

- een nicheproduct dat precies goed genoeg is

Dat is geen overgangsfase naar “het echte werk”. Dat is zelf echt werk.

En misschien wel een gezondere categorie werk dan de reflex om alles direct groot te willen maken.

MENSELIJKE KWALITEIT VERSCHUIFT NIET NAAR DE ACHTERGROND

Als je alle hype eraf haalt, blijft nog iets heel ouderwets overeind:

goede dingen worden nog steeds gemaakt door mensen die goed kunnen kijken.

Door mensen met:

- oordeel
- aandacht
- smaak
- discipline
- nieuwsgierigheid
- volharding

De machine versnelt. Maar dat maakt menselijk onderscheid niet overbodig.

Het maakt het zichtbaarder.

Want als veel meer mensen toegang krijgen tot vergelijkbare bouwkracht, wordt niet de ruwe mogelijkheid schaarser, maar de kwaliteit van keuzes.

DE BEHOEFTE AAN EIGEN ARCHIEF EN EIGEN GROND ONDER DE VOETEN GROEIT

Nog iets dat blijft: hoe meer je kunt maken, hoe belangrijker het wordt om te weten wat je bewaart.

Een vluchtig speelveld verleidt tot vluchtig gedrag. Je maakt iets. Probeert iets. Laat iets ontstaan. Gaat weer door.

Dat is prima, zolang iets echt tijdelijk is. Maar zodra iets waarde krijgt, wil je ook een eigen grondlaag.

Je wilt weten:

- wat bron is
- wat tussenversie is
- wat publicatie is
- wat je wilt meenemen
- wat je best kunt loslaten

Maken zonder geheugen is opwindend. Maken met geheugen is duurzamer.

DE BESTE MAKERS BOUWEN NIET ALLEEN OUTPUT, MAAR EEN PRAKTIJK

Misschien is dat wel de diepste les van deze beweging.

De mensen die hier op termijn het meest aan hebben, zijn zelden de mensen die elke nieuwe tool als eerste proberen. Het zijn vaker de mensen die langzaam een eigen praktijk opbouwen.

Een praktijk met:

- vaste vragen

- een herkenbare werkstroom
- een manier van noteren
- een manier van kiezen
- een manier van testen
- een manier van publiceren

Dan wordt vibecoding geen los trucje, maar onderdeel van hoe je werkt. Juist daardoor overleeft het makkelijker de hypefase.

OVERVLOED VRAAGT OM SELECTIEVER LEVEN

Als bijna alles makkelijker maakbaar wordt, ontstaat er een nieuw probleem: te veel mogelijke dingen om aan te beginnen.

Het is een luxeprobleem, maar wel een echt probleem.

Niet elk maakbaar idee is een goed idee. Niet elke snelle proef hoeft een project te worden. Niet elke werkende eerste versie verdient onderhoud.

Wie daar niet selectiever in wordt, verdrinkt alsnog.

Dus ook dit blijft: de kunst van het weglaten.

Niet alleen in interfaces. Ook in projecten.

DE HYPE VERDWIJNT, DE VERSCHUIVING NIET

De hype verdwijnt altijd. De opwinding vlakt af. De taal wordt normaler. De gekte beweegt door naar iets nieuws.

Maar de onderliggende verschuiving blijft vaak gewoon bestaan en wordt dan pas echt zichtbaar.

Dat gebeurde met publiceren op het web. Met digitale fotografie. Met desktop publishing. Met video maken. Met muziekproductie.

Eerst was het spectaculair. Later werd het infrastructuur.

Ik vermoed dat iets soortgelijks hier ook gebeurt.

Niet omdat elke tool zal blijven, wel omdat de mentale stap al gezet is:

veel meer mensen weten nu dat software iets is waar ze dichterbij op kunnen zitten dan vroeger.

En dat verdwijnt niet zomaar meer.

WAT DIT BOEK UITEINDELIJK VERDEDIGT

Als ik eerlijk ben, gaat dit boek uiteindelijk over meer dan vibecoding alleen.

Het verdedigt een bepaald soort makershouding.

Niet wachten tot alles perfect duidelijk is. Niet afhankelijk blijven waar dat niet nodig is. Niet groter denken dan het werk vraagt. Niet verliefd worden op gereedschap. Wel leren kijken. Wel leren kiezen. Wel iets durven maken dat eerst nog niet bestond.

Vibecoding is daarvoor nu een sterke hefboom. Maar de houding zelf is breder en ouder.

HET RUSTIGE SLOT

Misschien is dat ook de juiste manier om af te sluiten. Niet met een grote voorspelling, niet met de claim dat iedereen nu softwaremaker wordt en ook niet met een lofzang op de nieuwste generatie tools.

Veel belangrijker is dit: voor veel meer mensen dan voorheen hoeven ideeën niet meer in de idee-fase te sterven.

Niet alles wat maakbaar wordt, hoeft gemaakt te worden. Niet alles wat gemaakt wordt, hoeft te blijven bestaan. Niet alles wat blijft bestaan, hoeft groot te worden.

Maar iets wat ooit alleen een gedachte was, kan nu veel vaker een eerste vorm krijgen.

Voor een maker is dat een groot verschil.

WAT MOET BLIJVEN HANGEN

Als je na dit boek maar één ding meeneemt, laat het dan dit zijn:

Als de hype weg is, blijven niet de toolnamen over.

Wat overeind blijft: veel meer mensen kunnen software benaderen als iets wat zij mede kunnen vormen.

En dat maakt de wereld, ondanks alle ruis, een stuk rijker aan mogelijke makers.

MAKER MANUAL 01: VIBECODING - LICENTIE

Dit manuscript valt onder de Creative Commons-licentie CC BY-SA 4.0.

Dat betekent in gewone taal:

- delen mag

- hergebruiken mag
- bewerken mag
- ook commercieel gebruik is toegestaan

Voorwaarden:

- noem Erwin Blom als bron
- vermeld dat het manuscript met hulp van AI is omgezet tot Maker Manual
- deel afgeleide versies onder dezelfde licentie

Officiële licentietekst:

Creative Commons Attribution-ShareAlike 4.0 International